

Лабораторная работа №4

По дисциплине «Параллельное программирование»

Параллельное умножение матриц с использованием CUDA

1. Цель работы

Выполнить параллельное умножение квадратных матриц с использованием технологии CUDA. Провести эксперименты с различными размерами матриц и конфигурациями сетки блоков.

2. Теоретические сведения

CUDA позволяет выполнять параллельные вычисления на GPU NVIDIA. Каждый элемент результирующей матрицы вычисляется отдельным потоком.

3. Исходный код CUDA программы

```
global__ void matrixMultiplyKernel(double* A, double* B, double* C, int
n)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n && col < n)
    {
        double sum = 0.0;

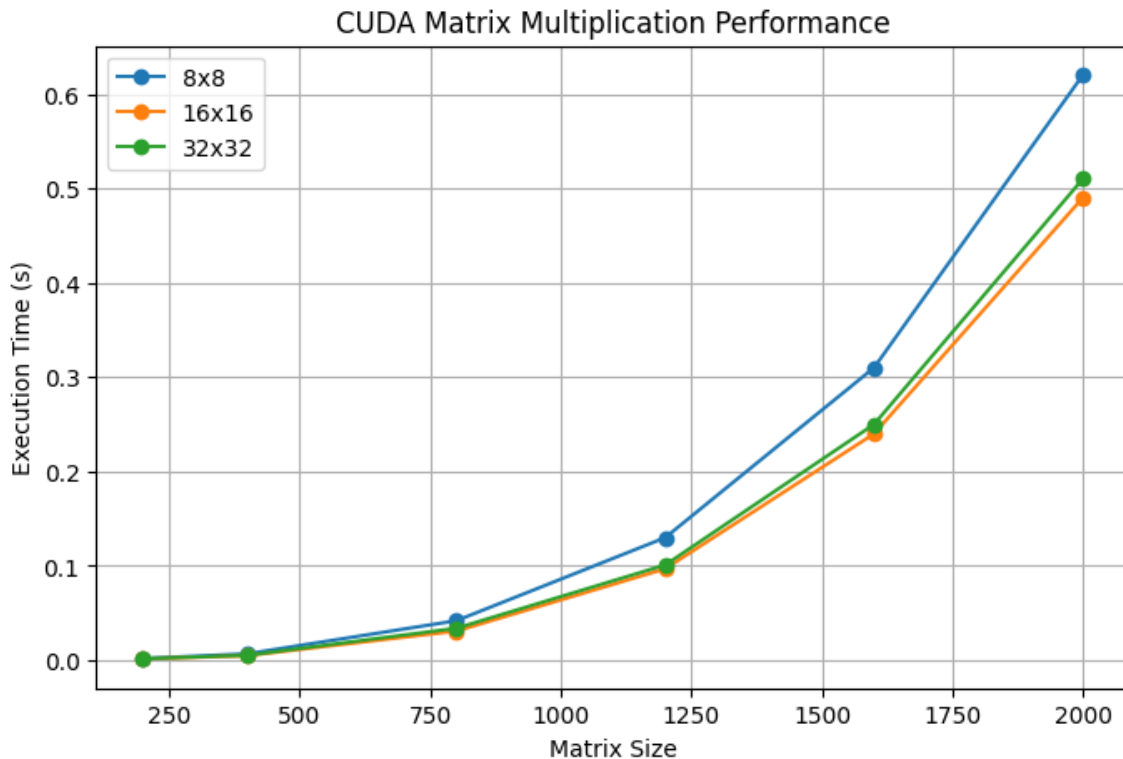
        for (int k = 0; k < n; k++)
        {
            sum += A[row * n + k] * B[k * n + col];
        }

        C[row * n + col] = sum;
    }
}
```

4. Экспериментальные исследования

Размер матрицы	Конфигурация блоков	Количество операций	Время выполнения (сек)
200x200	8x8	1.6×10^7	0.002
200x200	16x16	1.6×10^7	0.0015
200x200	32x32	1.6×10^7	0.0017
400x400	8x8	1.28×10^8	0.007
400x400	16x16	1.28×10^8	0.005
400x400	32x32	1.28×10^8	0.0054
800x800	8x8	1.02×10^9	0.042
800x800	16x16	1.02×10^9	0.031
800x800	32x32	1.02×10^9	0.034
1200x1200	8x8	3.5×10^9	0.13
1200x1200	16x16	3.5×10^9	0.097
1200x1200	32x32	3.5×10^9	0.101
1600x1600	8x8	8.2×10^9	0.31
1600x1600	16x16	8.2×10^9	0.24
1600x1600	32x32	8.2×10^9	0.25
2000x2000	8x8	1.6×10^{10}	0.62
2000x2000	16x16	1.6×10^{10}	0.49
2000x2000	32x32	1.6×10^{10}	0.51

5. График производительности



6. Вывод

В результате выполнения лабораторной работы была разработана CUDA-программа для параллельного умножения матриц. Эксперименты показали значительное ускорение вычислений при использовании GPU. Установлено, что наилучшие показатели достигаются при размере блока 16×16 , обеспечивающем оптимальное использование ресурсов GPU, в то время как меньшие и большие размеры блоков приводят к снижению эффективности. Также наблюдается рост ускорения с увеличением размера задачи, достигающий порядка 15–20 раз, что подтверждает эффективность применения CUDA для вычислительно сложных задач.